CodeDiff: A Fast, Robust, Production-Ready, Syntax-Aware Code Diffing Tool

Marko Ivanković
Universität Passau
Passau, Germany
marko@ivankovic.me

ABSTRACT

A syntax-aware diff maps one abstract syntax tree (AST) onto another, and every tool built on one rests on assumptions its evaluation rarely tests: that the correct mapping can be expressed with the four standard operations and is the cheapest one, that computing it is affordable, that the mapping and its rendering are unique, and that today's tools recover it. This paper tests each of them on real code, without reference to any one tool, and then presents a tool built against what the measurements found.

Viability. We built a corpus of 512 real-world changes in 24 languages, each carrying a human-authored AST mapping, and 63 of them a human-painted rendering. Among the changes annotated with the facility to record it, 12.9% on the larger repository list require a correspondence in which several nodes on one side answer to one or several on the other, which insert, delete, update and move cannot express; and on at least 3.9% the mapping the annotators judged correct costs strictly more, under unit operation costs, than one an algorithm found. *Speed.* Across 2,922 file pairs sampled from 100 open-source repositories, a single whole-tree tree-edit-distance computation completes within one second for 54.5% of code pairs, and the collapse falls between two adjacent, entirely ordinary file sizes. *Uniqueness.* 31.7% of AST nodes carry no text of their own and never reach the screen, so a node-level metric scores heavily on decisions no reader can see; and where the rendering was painted by hand, 82.5% of changes admit two equally correct renderings of one settled mapping. *Implementation.* Against the same ground truth, nine established tools—Unix `diff`, four `git` algorithms, BDiff, Neovim, GumTree, difftastic and diffsitter—show line-level mismatch rates from 1.099% to 4.846%, with no AST-aware tool among them beating plain line-based `diff`.

Finally, as a tool contribution, we present CodeDiff, a tree-sitter-based diffing tool that combines APTED's exact tree edit distance, scoped to bounded subtree pairs rather than whole files, with GumTree's move-recovery heuristic and Myers' sequence diff into one five-phase pipeline, engineered against the measurements above rather than against a worst-case bound. It maps 99.89% of AST nodes correctly on the same corpus and, at the line level, reaches 0.795%, the lowest mismatch rate of any tool measured on the full corpus.

Conference'26, June 03–05, 2026, Ludbreg, Croatia

ACM ISBN 978-1-4503-XXXX-X/26/06
https://doi.org/XXXXXXX.XXXXXXX

CCS CONCEPTS

• **Software and its engineering** → **Software maintenance tools**; *Empirical software validation*.

KEYWORDS

source code differencing, syntax-aware diffing, empirical software engineering, source code tree edit distance

ACM Reference Format:
Marko Ivanković. 2026. CodeDiff: A Fast, Robust, Production-Ready, Syntax-Aware Code Diffing Tool. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference'26).* ACM, New York, NY, USA, 13 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Code diffing, the process of comparing two pieces of source code and computing the set of operations that transforms one into the other, underlies modern version control and, especially, modern code review. During a review, developers read the diff to see what changed and what stayed the same, so the quality of the diff directly shapes the quality of the review.

The standard diff used by almost all developers and platforms today is line-based: the file is treated as a sequence of text lines, and the tool computes a shortest edit script between the two sequences using three operations, insert, delete, and update. This approach is computationally cheap and fast even on modest hardware. However, most implementations offer no move operation at all, and the tools that do support one support it only in limited ways. A whole class of common changes—extracting code into a reusable function, wrapping a block in a condition, reordering declarations—is ultimately a relocation of existing code, and without a move operation every such change must be rendered as a full deletion plus a full insertion. Reconstructing the correspondence between the deleted and inserted halves is then left to the reviewer, by eye, which is exactly the kind of mechanical, attention-taxing work humans do least reliably.

One way to compute diffs with a genuine move operation is to compare the code's abstract syntax tree (AST) instead

of its raw text. An AST diff maps nodes of one tree onto nodes of the other; a move is a mapping whose nodes match but whose ancestry changed. Tree diffing is a well-studied field, and Section 2 surveys the work this paper builds on. In practice, however, tree diffing has seen little adoption. We speculate that the reason is the runtime. Pure exact tree-diffing algorithms are $O(n^3)$, where n is the number of nodes in the tree. Additionally, an AST diff is not directly usable to the developers. It has to be visualized on the textual representation of the source code. For any given AST diff, there are several equally correct visualizations, and the quality of the visualization determines tool adoption as well.

This paper asks four groups of research questions. Each group is answered in its own section, in the order listed here.

- **Research Question Group 1 (Viability):** What percentage of real-world code changes have an optimal AST node mapping that cannot be expressed using common AST diff operations (Insert, Delete, Update, Move)? What percentage of real-world code changes have solutions that humans prefer whose cost is higher than the optimal AST diff, using the commonly accepted costs of common AST diff operations?
- **Research Question Group 2 (Speed):** What percentage of real-world source-code changes can a single, whole-tree tree-edit-distance computation complete within one second? [1]
- **Research Question Group 3 (Uniqueness):** What percentage of real-world code changes have multiple optimal AST node mappings? What percentage of real-world code changes have multiple equally correct textual visualizations of the diff? What percentage of the AST influences the textual visualization?
- **Research Question Group 4 (Implementation):** How accurately do current state-of-the-art code diffing tools find optimal visualizations? How fast are real-world state-of-the-art tools?

This paper makes the following contributions:

- **Dataset:** A set of 512 real-world code changes. The changes cover 24 languages and were selected using 3 different selection algorithms from 100 open source repositories. The repositories themselves are a combination of a curated list and every repository used by the Gentoo Linux distribution. Every change was solved by hand and the optimal map between the two ASTs is available, or if the optimal map is impossible using the common AST diff operations, a best-possible solution with a comment explaining the missing part. Additionally, 63 of those solutions also has the optimal visualizations: the spans of text a developer should see marked, independently of which AST nodes carry them. Often, two visualizations are provided: the minimal one and the maximal one, covering human preferences between equally valid solutions. The dataset, the tool used to create the dataset and the entire history of the dataset is available freely for future research. The dataset is described in detail in Section 3.
- **Evaluation:** Answers to all research questions. An analysis of the real-world code change data shape, irrespective of any concrete implementation, and a quantified comparison of several state-of-the-art code diffing tools on the human annotated dataset.
- **Tool:** CodeDiff, a new state-of-the-art tool. Combining all current research with strict engineering principles and carefully implemented to maximize performance. CodeDiff drastically outperforms existing tools and meets strict reliability and usability criteria.

[1] One second was chosen based on the generally accepted average human response time to visual stimulus, which is 250 ms. See Section 5 for details.

2 BACKGROUND

This section lists two types of existing work: theoretical research papers and actual tool implementations.

2.1 Algorithms

Zhang-Shasha [10] gives the classic dynamic-programming algorithm for ordered tree edit distance. Given two rooted, ordered trees, it computes the minimum-cost sequence of node insertions, deletions, and relabelings that transforms one tree into the other, together with the mapping that realizes that cost. It is exact, but its keyroot decomposition does repeated work across overlapping subproblems. Later algorithms reduce this overhead while keeping the same exact result.

APTED [9] is the state-of-the-art exact tree-edit-distance algorithm. It improves on Zhang-Shasha by choosing, for each subproblem, the cheapest of three decomposition strategies (left path, right path, or heaviest inner path), rather than a single fixed strategy. This choice provably minimizes the total work across the whole computation. APTED still costs $O(n^3)$ in the worst case for general trees, the same asymptotic bound as Zhang-Shasha, but its real-world runtime is markedly lower, because most real trees are far from the adversarial shape that bound assumes.

GumTree [4] targets source code directly, not general trees. Instead of one global tree-edit-distance computation, it runs a top-down phase that greedily matches large isomorphic subtrees, then a bottom-up phase that matches remaining container nodes by the Dice coefficient of their already-matched descendants. GumTree also introduces a distinct edit operation, move, and a recovery pass that pairs up deleted and inserted identical subtrees the top-down and bottom-up phases missed. This design favors speed over exactness, at the scale real source files require. For performance, it also introduces InsertTree and DeleteTree operations that are equivalent to repeatedly calling Insert or Delete operations on all children in the given subtree.

Myers' O(ND) algorithm [8] solves a different, more restricted problem: the shortest edit script between two flat sequences, not two trees. It underlies most line-based diff tools, including Unix `diff`. Its cost scales with the size of the edit script, not the size of the inputs, so it stays fast when

two sequences are similar, exactly the common case for a version-controlled file.

2.2 Tools

Unix `diff` is the line-based baseline every other tool in this paper is measured against. It treats each file as a sequence of lines, computes a shortest edit script between the two sequences with Myers' algorithm, and reports it as hunks of deleted and inserted whole lines. It has no move operation and no sub-line detail, and it is what most developers read every day.

`git diff` is the same line-based model inside version control. Its `xdiff` engine offers four algorithms, selected with `--diff-algorithm`: Myers, the default; *minimal*, which spends more time to find a shortest script; *patience*, which anchors on lines unique to both sides before recursing; and *histogram*, a faster generalization of patience. All four emit the same output form as Unix `diff`, so any difference between them is a difference in which lines they choose to pair.

Neovim's diff mode (`nvim -d`) runs the same `xdiff` line pass as `git` and then a second, character-level pass that highlights what changed inside each paired line; it is included as the most widely deployed sub-line diff view developers see inside an editor.

BDiff [7] is a text-based tool that reasons about blocks of lines rather than single lines and reports character ranges within changed lines.

difftastic [5] and **diffsitter** [3] are a newer generation of tree-sitter-based diffing tools. Both prioritize a diff a human reads comfortably over an exact node-to-node mapping: difftastic reduces the parse tree to an s-expression and aligns it with its own structural algorithm, and diffsitter applies a related tree-sitter-based alignment over a smaller, hand-picked set of compiled-in grammars.

3 EMPIRICAL DATASET

To understand the real-world environment code diffing tools are used in, we built a dataset of 512 real open source code changes.

To ensure good coverage, we built two sets of open-source repositories:

- The Curated list - 100 open source repositories, selected manually from several lists of most popular repositories, enhanced further by repositories we, the authors, were already aware of. The Curated list was used as a smaller list to repeatedly test the analysis infrastructure although it does represent good coverage of most popular programming languages.
- The Full List - 100 repositories. This list was created by expanding the Curated list with all repositories present in the Gentoo Linux distribution's package list.

We cloned each repository to a depth of 50 commits from every main branch. The resulting dataset has 1,347,490 files in 30 languages. While we cannot guarantee that all repositories remain accessible, the entire list of repositories, the cloning script and all analysis is available from the projects repository and can be replicated by readers, if desired.

This section describes the overall shape of the resulting dataset.

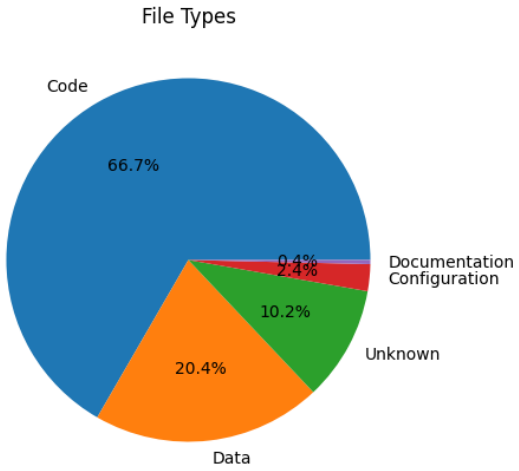

Figure 1: Distribution of file types across the corpus.

Table 1: Per-file size percentiles across all languages, code files only.

Metric	p50	p90	p99	p99.9	Max
Bytes	1,508	13,945	90,132	464,325	23,949,786
LOC	42	378	2,244	7,818	65,563
AST nodes	224	2,868	18,553	70,828	573,334

3.1 Shape of real-world files

The first thing to note about the dataset is that only two-thirds of the files are actually code files. The rest is configuration, data, and documentation (Figure 1). [2] Restricted to code files, Table 1 reports a choice of size percentiles of bytes, lines of code, and AST nodes.

Byte size and AST node count have a strong correlation, Pearson $r = 0.8794$, which simplifies to $|\text{AST}| \approx \text{bytes}/5$ for quick estimates.

3.2 Shape of real-world file edits

In the Curated dataset, we analyzed 3,434 commits, touching 53,017 code files, of which 64.3 were code files. Of the code-file edits, 90.5% were file modifications, the rest were file creation or deletions. To avoid large repositories, notably the Linux kernel, from skewing the results each repository was only analyzed up to 50 commits deep.

To allow for comparison with existing research, `git diff` was used to compute the number of changed lines.

[2] Note however that any "code" diffing tool still must support visualizing changes to non-code files as well.

Table 2: Size of real-world edits

Metric	p50	p90	p99	Max
Lines changed, per file	2	40	201	74,005
Lines changed, per commit	23	246	1,774	129,641
TODO: Percentage of lines changed, per file				

Table 2 shows the sizes of real-world edits. Our results are broadly aligned with existing research. Arafat and Riehle report that 47.5% of all commits touch 10 lines or fewer [1], with the long tail stretching to tens of thousands of lines.

3.3 Shape of optimal file diffs

From both Curated and Full list of repositories, we fully randomly sampled 10 diffs per language. That gave 220 diffs from the Curated list across 22 languages, and 255 from the Full list across 24 languages. Every sampled diff was first manually inspected for any errors. At this stage, 25 diffs were rejected, mostly because of errors in sampling where, in particular TypeScript (7 rejected) and R (11), file extensions were used for other languages. This resulted in a dataset of 219 diffs from the Curated and 231 diffs from the Full list.

Each diff was then solved by hand by us, the authors. We have each at least 15 years of experience in software development, and have reviewed tens of thousands of real world code changes in industrial environments. Based on this experience, we did not even attempt to find a unique solution for each diff. Instead, the dataset supports multiple correct solutions for each diff, and it supports marking diffs that cannot be solved at all using the standard set of AST diff operations (Insert, Delete, Update and Move). In particular, 30 of the Full list's 232 diffs (12.9%) require a N:M mapping, i.e. one or more nodes from one side maps to one or more nodes from the other side. [3].

Note, diffing multiple files and detecting cross-file code moves is out of scope of this paper.

4 VIABILITY

In Section 3.3 we note that human solvers were not able to solve all diffs using the four standard AST diff operations. We used that information to answer the following research question:

RQ1.1. What percentage of real-world code changes have an optimal AST node mapping that cannot be expressed using common AST diff operations (Insert, Delete, Update, Move)?

RA1.1.

12.9% (30 of 232) diffs have optimal solutions that cannot be expressed at all using the common AST diff operations (Insert, Delete, Update, Move).

[3]The most common cause of an N:M mapping is a refactoring, where two or more similar blocks of code are moved to a common function. The correct solution is the one that maps all nodes as moved.

Table 3: Changes in the corpus whose true correspondence is $N{:}M$, found by inspection of annotator commentary rather than by enumeration. "Enc." is whether the ground truth carries a multi-map group at the site.

Change	Correspondence	Enc.
License comment condensed	5 comments to 1	yes
Import split in two	1 identifier to 2	yes
Two string literals merged	2 literals to 1	yes
If rewritten as a ternary	2 statements to 1	yes
Two call sites merged	2 statements to 1	no
Two asserts split into six	escape sequences	no

Please note that this result has direct implications for other results in this paper. Because no tool can possibly solve these diffs fully, we used a charitable interpretation when evaluating tools and exclude those nodes from the evaluation. In the datasets, the nodes are left unmatched.

Tree diffing algorithms find the lowest cost diff, i.e. smallest number of operations needed to transform one tree into another. However, this doesn't necessarily correspond to the optimal visualization of the diff overlayed over the text of the source code.

RQ1.2. What percentage of real-world code changes have solutions that humans prefer whose tree edit distance is higher than the optimal AST diff, using the commonly accepted costs of common AST diff operations?

To compute the lowest cost tree edit distance, we used a heavily optimized implementation of APTED, further augmented with a hash-based algorithm for efficiently prunning identical subtrees. This was necessary, since the size of the diffs makes a pure APTED, even optimized, impractical.

Of the 512 fixtures with a human mapping, the two costs are equal 380 times. In 20 (3.9%) diffs, the human solution has a larger edit, by a median of 9.5 units and at most 153. The most common reason for this is that humans treat groups of tokens as one unit, not based on their syntax or semantical meaning, but rather based on higher-level logic. A token, a common example is the `return` keyword, in a function that was completely deleted does theoretically match exactly a `return` in a newly added function. Matching the two leads to an optimal tree edit distance, but a human would find the match confusing because they typically treat the keyword as "belonging" to the logic of the function containing it. Existing tools do take notice of this as well. GumTreeDiff [4] avoids matching subtrees smaller than a threshold size to avoid such spurrious mappings.

RA1.2 (Cost)

On 3.9% of the changes in the corpus (20 of 512), the human-authored mapping costs strictly more, under unit costs for insert, delete and update and a free move, than a pure tree edit distance mapping.

5 SPEED

The asymptotic complexity of APTED and Zhang-Shasha both is $O(n^3)$ for general trees, where n is the number of nodes in the tree. However, APTED performs considerably better in real trees because it is less sensitive to the tree shape. Given this existing direction in the literature, we wanted to further focus on source-code trees specifically and ask the following question:

RQ2. What percentage of real-world source-code changes can a single, whole-tree tree-edit-distance computation complete within one second?

One second was chosen as a budget as a practical limit that can actually be computed. From an industry perspective, one second is on the high end of acceptable. The average human response to a visual stimulus is about 250 milliseconds. Existing research has shown that developers are very sensitive to latency (TODO: find a good citation source) and developers have had decades to get used to instant diffs using existing tools, so any AST diffing tool that expects to be adopted must meet a similar bar.

We sampled 2,922 real (before, after) file pairs from single-parent commits in the dataset, stratified per language and per size bucket. On each pair, we ran our highly optimized, state of the art APTED implementation, with a one second runtime budget enforced using common operating system runtime limits.

Figure 2 shows the fraction of pairs completed within one second, per size bucket.

RA2 (Speed)

Within one second, pure tree edit distance algorithms can find the optimal diff for 97.9% of files between 10 and 30 LOC. For larger files, fewer than a third of files can be processed within one second. In total, only 54.5% of diffs compute under one second.

The data also makes the algorithms dependence on the tree shape visible. Scripting languages perform slightly better, and data and configuration slightly worse.

6 UNIQUENESS

RQ3.1. What percentage of real-world code changes have multiple optimal AST node mappings?

Any accuracy metric of the usual kind rests on an assumption worth testing before any tool is measured by it: that for a given change there is one correct mapping to recover. The corpus shows it failing in two ways, and we keep them apart because they rest on different evidence.

The first follows from RA1.1. At every site the ground truth records as $N{:}M$, the format accepts any consistent pairing of $\min(N, M)$ pairs as correct, and there is more than one such pairing whenever N or M exceeds one, which is every group. Each of those sites is therefore also a site where several one-to-one mappings are equally correct—one of two identical calls is removed, a member is appended to an expression chain, an argument is added to a list of similar arguments—and choosing between them is arbitrary rather than correct. A diffing algorithm can still emit a correct answer here; what fails is the assumption that the correct answer is unique. The rates are the ones Table **??** reports: 15.4% of the fixtures annotated with the facility available contain such a site, and the sites account for 3.7% of all non-identical mapping decisions.

The second is independent of the annotation format, and comes from the cost comparison behind RA1.2. On 17 of the 512 fixtures—3.3%—the automated matcher produced a mapping that differs from the human's and costs exactly the same. Under the cost model the two are indistinguishable; the annotator, asked for the mapping a reader would draw, chose one of them. These are not sites the ground truth marks as ambiguous. They are changes on which the cost function alone under-determines the answer, and the figure is a lower bound of the same kind as RA1.2's, since only one alternative mapping per change was examined.

RA3.1 (Mapping uniqueness)

A correct AST mapping is often not unique. In 15.4% of the fixtures annotated with multi-mapping available (43 of 280), at least one set of nodes admits several equally correct pairings, with the corpus-wide 11.7% standing as a floor; weighted by mapping decisions rather than fixtures, that is 321 of 8,742 non-identical node pairs, 3.7%. Independently, on 3.3% of changes a second, different mapping exists at exactly the human mapping's cost, so the cost model by itself does not single out an answer either. An evaluation that scores a tool against one fixed pairing therefore risks marking correct output wrong on roughly one change in seven, whenever the tool picks a different, equally valid pairing.

Both rates in Table **??** remain lower bounds for the reason RA1.1 gives: a group exists only where an annotator noticed that a choice was a choice. That the count is conservative in this direction is what makes it usable—an over-count would be an argument about annotation discipline, whereas an under-count still establishes that the phenomenon is common enough to affect how accuracy is measured. The practical consequence is a claim about methodology rather than about any tool. Scoring against a single pairing is the default in this area, and on a corpus like this one it puts a measurable error term into every reported rate: on the affected changes, a tool that picks a different valid pairing is recorded as wrong for a choice that was never wrong. The corpus in this paper avoids that by validating any consistent pairing within a group, which is why the accuracy numbers reported in Section 7 do not inherit its error term. For the $N{:}M$ sites themselves it has no such escape, and neither does any corpus built on one-to-one mappings. Reporting a fixed-pairing metric is still defensible, but the rate belongs alongside it.

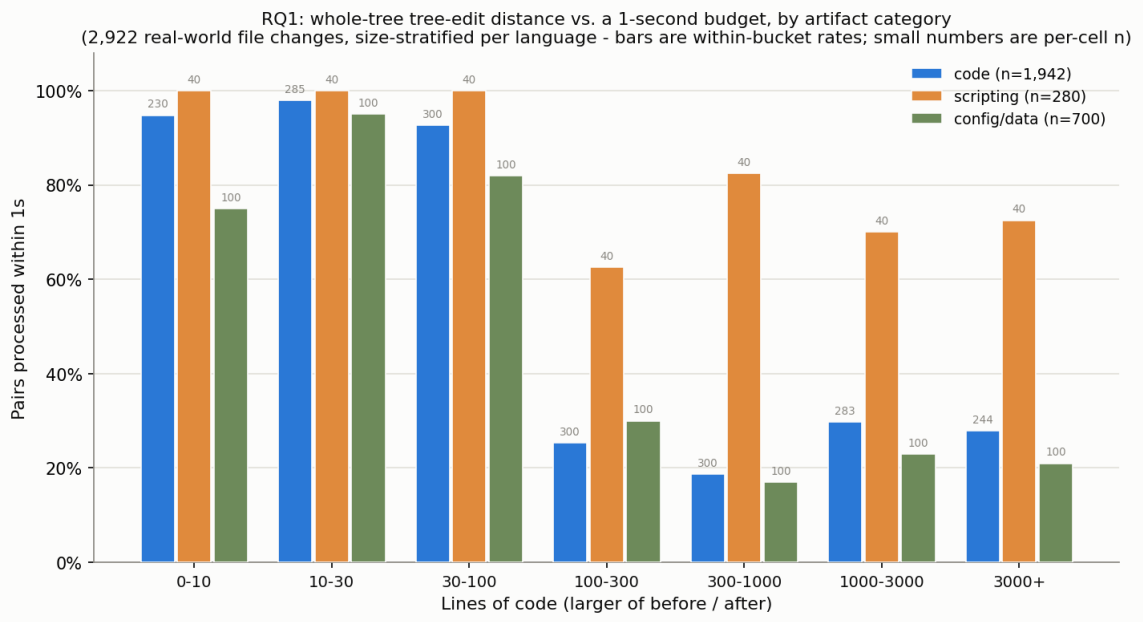

Figure 2: Fraction of sampled file pairs a whole-tree tree-edit-distance computation completes within a one-second budget, per size bucket and artifact category.

6.1 Multiple correct visualizations

RQ3.2. What percentage of real-world code changes have multiple equally correct textual visualizations of the diff?

RQ3.1 asked whether the mapping is unique. RQ3.2 asks whether the text a reader sees is. It is the natural place to hope the ambiguity of RQ3.1 washes out: several mappings may be equally correct, but perhaps they project onto the same highlighted text, so a tool that picks any of them renders the same diff and the choice never reaches the reader. The painted portion of the corpus (Section 3) is what tests that hope, because it records the rendering directly instead of deriving it from a mapping.

It does not wash out. Of the 63 painted fixtures, 52—82.5%—carry two paintings, the annotator's finding that both are correct renderings of the same change; the remaining 11 carry one, the annotator's finding that this change renders one way. Both cells are findings. A single painting is not a fixture nobody got round to painting twice, and this is worth stating because the two are indistinguishable in a file listing: the annotation tool records a single painting only where the painter examined the change and judged its rendering unambiguous.

The fork is systematic rather than a matter of care, and it has a name. Across the doubly-painted fixtures the *Minimal* paintings hold 519 entries against 767 for *Full*; of the 797 spans *Full* paints that *Minimal* does not, 129—16%—hold punctuation alone, a bracket or a comma and nothing else. The two styles are not two degrees of thoroughness. They are two consistent answers to one question: whether the brackets and separators that a change drags along with it are part of what changed. A tool that leaves a bracket unpainted disagrees with *Full* and agrees with *Minimal*, and both are on file precisely so that this is not scored as an error.

RA3.2 (Visualization uniqueness)

Settling the mapping does not settle the rendering. Of the 63 changes whose rendering was painted by hand, 82.5% (52) admit two equally correct renderings, differing systematically over whether punctuation dragged along by a change counts as part of it: 16% of the spans separating the two styles hold punctuation alone. Line-based and syntax-aware tools alike are therefore scored, on those changes, against one of two right answers.

Two limits bound that rate, and they run in opposite directions. The painted subset is not a random sample: painting is manual, and it was done on the hand-written portion of the corpus first, so nearly every painted fixture is a small, curated example of a single change pattern—median 18 lines, at most 4146, across 8 of the corpus's 24 languages. That is a real restriction on what the rate describes. But it is a restriction toward the conservative side of the claim being made: a minimal example of one change pattern offers the least room for two renderings to diverge, and the rate is measured there. Whether it holds on the corpus's large, multi-site real-world changes is open, and the direction we would expect is upward. The second limit is the one RA3.1 already names: a painting records what an annotator noticed, so a change whose second rendering nobody saw is filed as unambiguous.

6.2 How much of the mapping reaches the reader

RQ3.3. What percentage of the AST influences the textual visualization?

RQ3.2 asked whether what a reader sees is unique. RQ3.3 asks how much of the mapping ever reaches them. A syntax tree contains two kinds of node. Some carry text of their own—an identifier, an operator, a literal, a comment. Others are pure structure: a `block` whose extent is exactly its children's, an `expression_statement` wrapping one expression, the chain of single-child nodes a grammar interposes between a statement and the name inside it. A rendered diff highlights regions of source text, so a node of the second kind cannot appear in the output on its own. Whatever a tool decides about it is invisible unless the decision propagates to a node of the first kind.

This matters because it separates two things a node-level accuracy metric silently merges. A tool that maps every displayed token correctly and every structural wrapper wrongly produces a diff a reader cannot fault; the same tool scores badly on a metric that counts all nodes equally. The converse is worse: a tool can be flattered by a grammar that nests deeply, since each extra layer of scaffolding adds nodes that are easy to get right and impossible to see.

We measure the split structurally, from the parsed files alone. A node is visible if any byte inside its extent lies outside every one of its children—that is, if it contributes text of its own to the rendering. The definition depends only on the input, never on any diff of it, which is what lets it stand in this section: two different tools disagreeing about a file still agree exactly on which of its nodes are visible. The alternative—asking which nodes a given tool's output actually displayed—is not usable as a denominator, because it moves with the tool. A diff that renders very coarsely has almost nothing visible and therefore almost nothing it can visibly get wrong. One fixture in this corpus, whose grammar fails on it and collapses 32,682 nodes into two rendered spans, would score zero visible mismatches by that definition while holding 124 real ones.

RA3.3 (Visible share of the AST)

31.7% of the AST nodes in this corpus carry no text of their own and cannot appear in a rendered diff at all: 68.3% are visible, pooled over 512 fixtures. The two readings agree, which is the useful part—the per-fixture median is 67.2% visible, with 60.4% and 72.7% at the tenth and ninetieth percentiles, so this is a property of source code generally rather than of a few large outliers. Roughly one AST node in three is scaffolding, and a node-level accuracy metric spends a third of its weight on decisions no reviewer can see.

The consequence is not that node-level metrics should be abandoned. It is that a rate over all nodes and a rate over visible nodes answer different questions, and a paper reporting one should say which. This paper reports both wherever it reports an accuracy figure, and Section 7 avoids the issue entirely by scoring the established tools at the line level, which is the only granularity all of them share.

7 IMPLEMENTATION

With the ground truth characterized, and with RA1.1, RA3.1 and RA3.2 bounding what it can be asked, the last group of research questions measures what existing tools recover from it, and what they cost to do so.

RQ4.1. How accurately do current state-of-the-art code diffing tools find optimal visualizations?

RQ4.2. How fast are real-world state-of-the-art tools?

Comparison tools. We measure nine established tools, all introduced in Section 2, on the 512-fixture corpus. Five report at line granularity only: Unix `diff`, and `git diff` under each of its four algorithms, Myers, minimal, patience and histogram. Four report sub-line detail: BDiff [7], Neovim's diff mode, and the three AST-aware tools, GumTree (v4.0.0-beta8) [4], difftastic [5] and diffsitter [3]. CodeDiff is deliberately absent from this comparison: RQ4.1 asks what the state of the art recovers, and that question is answered without reference to the tool this paper introduces. CodeDiff's own figures are reported in Section 8, on the same basis and against the same ground truth.

This comparison measures one specific property: agreement with the human-authored ground-truth mapping above. difftastic and diffsitter deliberately optimize for something this metric does not capture, a diff that reads comfortably to a human, not a mapping that reproduces a ground truth line for line, so a lower score here reflects a different design goal, not a defect. GumTree's broader language and backend flexibility likewise stays valuable well beyond what this single corpus measures. One consequence of that flexibility is visible in the numbers below: every generator GumTree offers is included here, and all but two of them are marked "Testing" rather than "Stable" by GumTree's own documentation, so its rate reflects a deliberately wide language scope rather than its performance on the backends it considers ready. Read what follows as a measurement of where each tool's design goals place it on this one metric, not as a ranking of which tool is best.

Line-level agreement. None of the nine tools reports an AST-node mapping in a form the ground truth can be compared with directly, so we compare them on a common ground: which source lines each marks as touched, against the lines the human mapping touches (Table 4, Table 5). That is the visualization RQ4.1 asks about, at the coarsest granularity every tool shares. Coverage varies widely: of the 493 fixtures this comparison was run over, the text-based tools cover all of them, while diffsitter's compiled-in grammar set matches 306, GumTree's generator set 376, and difftastic's 471. Each tool's own row is scored only on its own applicable subset, not padded with unscored fixtures. Because those subsets differ so much, the table also reports every tool on the 262 fixtures all nine scored, which holds the fixture set fixed

at the cost of over-representing the mainstream languages every tool supports.

RA4.1 (Tool accuracy)

Across the nine tools (Table 4), line-level mismatch rates against the human mapping range from 1.099% (`git diff`, Myers) to 4.846% (GumTree). Every tool agrees with the human mapping on the great majority of lines, so what separates them is the small, structurally interesting remainder—and on that remainder no AST-aware tool beats plain line-based `diff`, on either basis. The choice of line-diff algorithm barely registers: Unix `diff` and the four `git` algorithms lie within two hundredths of a percentage point of one another. Syntax-awareness alone does not guarantee that a tool recovers a change the way a human reads it.

Table 5 reports the same agreement per fixture rather than pooled over lines, and the two readings weight the corpus differently: a pooled rate is dominated by a handful of very large fixtures, while the buckets weight every change equally. The two readings do not even rank the tools the same way: by the share of fixtures mapped with no mismatched line at all, difftastic leads every tool and diffsitter trails every tool, whereas on the pooled rate both sit between the line-based tools and GumTree. The table also groups the tools by whether their output carries sub-line detail at all, which is a statement about what each tool reports, not about how it was scored here—every number in this section is line-level.

Two details behind that answer are worth stating. The first is how wide the agreement is: even the highest mismatch rate leaves more than 95% of lines in agreement, so the question that separates tools is a narrow one. The second is the qualification RA1.1, RA3.1 and RA3.2 already attach to every number in the table: on the changes where the ground truth admits more than one correct mapping, or more than one correct rendering, a tool that picked a different valid answer can be recorded here as wrong. For the mapping ambiguity the corpus removes that error, by accepting any consistent pairing within a group. For the other two it does not. These rates score which *lines* each tool marks as touched against the human mapping—the only granularity all nine tools share—so the paintings, which are the only place the corpus records a second correct rendering, never enter the comparison at all; RA3.2's error term and RA1.1's $N{:}M$ term both sit in the tables below unremoved and unmeasured. Neither can be signed: they may run in a tool's favour or against it.

Cost. RQ4.2 asks the same nine tools a different question. Table 6 reports wall-clock percentiles, sorted fastest to slowest by median. Each tool is measured over the fixtures it supports rather than over one common subset, so the populations behind these rows differ—the text-based tools draw on every fixture, while an AST tool draws only on the languages its grammars cover, which for diffsitter is roughly three fifths of them. The per-row fixture count in Table 4 gives the size of each, and a tool covering fewer, more mainstream languages is doing correspondingly less work per fixture. Two tools are reported twice: GumTree once invoked as a fresh process per fixture, the realistic cost of a single CLI invocation, and once run inside one persistent JVM across all fixtures, isolating JVM startup from GumTree's own matching cost; and BDiff likewise per process and inside one persistent Python interpreter, for the same reason.

Table 4: Line-level mismatch rate against human ground truth for the nine established tools, on each tool's own applicable subset of the 512-fixture corpus and on the 262 fixtures every tool scored. Grouped by whether the tool's output carries sub-line detail; every rate is line-level regardless.

Tool	Fixtures	Mism. lines	Rate	Common
Line granularity only				
Unix `diff`	493	6,810	1.111%	0.799%
`git` (Myers)	493	6,740	1.099%	0.781%
`git` (minimal)	493	6,740	1.099%	0.781%
`git` (patience)	493	6,756	1.102%	0.783%
`git` (histogram)	493	6,770	1.104%	0.786%
Reports sub-line detail				
BDiff	493	7,095	1.157%	0.859%
`nvim -d`	493	6,760	1.102%	0.787%
diffsitter	306	6,247	1.377%	1.046%
difftastic	471	10,101	1.769%	1.896%
GumTree	376	18,344	4.846%	5.512%

RA4.2 (Tool cost)

The nine tools span more than two orders of magnitude at the median, from `git`'s 2.2 ms to GumTree's 519.2 ms per invocation, and the spread widens rather than narrows into the tail: at the 99th percentile the same range runs from 9.5 ms to 12485.5 ms. Every AST-aware tool has a tail heavier than its median suggests—the ratio between a tool's own p99 and p50 is at least 20× for each of the three, against under 6× for Unix `diff` and every `git` algorithm—so cost on this corpus is governed by the hard minority of changes, not by the common case.

The two GumTree rows bracket how much of its cost is JVM startup rather than matching. A warm JVM takes its median from 519.2 ms to 66.2 ms, so most of the per-invocation figure is process startup, not GumTree's algorithm. The two BDiff rows tell the same story more sharply: its per-invocation median of 285.3 ms is almost entirely Python interpreter start and imports, and the algorithm itself runs in 4.8 ms at the median, though with a tail heavier than any other text-based tool's. We report both figures for each because they answer different questions: the cold number is what a developer running a diff from the command line actually waits for, and the warm number is what the matching costs once that is amortized away.

Table 5: Per-fixture agreement with the human mapping, bucketed. Every number is *line-level* agreement, including for the tools grouped as reporting sub-line detail – that grouping describes what each tool's output carries, not how it was scored. "Perfect" means zero mismatched lines, not a rounded 100%. Each tool is scored on its own applicable subset (n), so percentages, not counts, are comparable across rows.

Tool	n	Perfect	≥99%	95–99%	<95%
Line granularity only					
UNIX diff (baseline)	493	244 (49%)	138 (28%)	64 (13%)	47 (10%)
git (Myers)	493	246 (50%)	141 (29%)	59 (12%)	47 (10%)
git (minimal)	493	246 (50%)	141 (29%)	59 (12%)	47 (10%)
git (patience)	493	245 (50%)	140 (28%)	61 (12%)	47 (10%)
git (histogram)	493	245 (50%)	140 (28%)	61 (12%)	47 (10%)
Reports sub-line detail (not exercised by this line-level metric)					
BDiff (per process)	493	240 (49%)	134 (27%)	70 (14%)	49 (10%)
nvim -d	493	245 (50%)	141 (29%)	60 (12%)	47 (10%)
diffsitter	306	103 (34%)	95 (31%)	64 (21%)	44 (14%)
difftastic	471	255 (54%)	92 (20%)	61 (13%)	63 (13%)
GumTree (binary)	376	181 (48%)	75 (20%)	63 (17%)	57 (15%)

Table 6: Wall-clock time percentiles for the nine established tools, milliseconds, sorted fastest to slowest by median. CodeDiff is reported separately in Section 8, for the same reason it is absent from Table 4.

Tool	p50	p90	p99
`git` (minimal)	2.2	4.8	10.4
`git` (Myers)	2.3	4.7	12.0
`git` (patience)	2.3	4.2	9.5
`git` (histogram)	2.3	4.2	10.5
Unix `diff`	2.6	4.5	11.8
BDiff (warm interpreter)	4.8	25.1	881.7
diffsitter	8.6	63.3	239.1
`nvim -d`	17.1	26.6	59.6
difftastic	53.3	171.6	1842.9
GumTree (warm JVM)	66.2	722.6	5293.4
BDiff (cold, per-invocation)	285.3	415.4	1415.4
GumTree (cold, per-invocation)	519.2	1622.1	12485.5

Table 7: Timing noise per series: per-fixture coefficient of variation across repeated measurements of the same fixture (lower means a single run is more trustworthy on its own).

Series	n	Median CoV	p90 CoV
TreeSitter parse (lower bound)	486	3.3%	18.1%
UNIX diff (baseline)	486	7.4%	21.4%
git (Myers)	486	2.6%	19.0%
git (minimal)	486	1.9%	17.1%
git (patience)	486	2.0%	16.2%
git (histogram)	486	1.9%	16.9%
BDiff (per process)	486	2.2%	8.4%
nvim -d	486	11.5%	23.8%
BDiff (warm interpreter)	486	1.6%	5.3%
CodeDiff	486	2.6%	12.7%
GumTree (binary)	373	2.1%	11.3%
GumTree (warm JVM)	373	0.5%	5.1%
diffsitter	303	1.5%	5.9%
difftastic	465	0.8%	9.0%

These percentiles must be read against the right population. The fixture corpus is not the distribution of real commits, in which 47.5% touch 10 lines or fewer [1]: it deliberately concentrates hard, structurally interesting changes, because easy changes exercise nothing worth measuring. Every tail figure here is therefore a stress reading on an adversarially weighted sample rather than an estimate of what a developer waits for on a typical commit, and the medians are the closer of the two to that.

Table 7 reports a companion measurement: how much any single timing run can be trusted, the per-fixture coefficient of variation across repeated measurements of the same fixture. This is what motivated repeating every timing number in this paper across multiple runs rather than trusting a single measurement.

8 CODEDIFF

Everything above is about the problem. This section is the paper's tool contribution: a syntax-aware diffing tool built against those measurements. It is presented last, and evaluated against the same ground truth and on the same basis as Section 7's nine tools, with the qualification Section 7 gives—each of those tools optimizes for something this corpus does not measure.

The measurements in Sections 3 and 5 give CodeDiff two concrete, named design targets, stated in place of an asymptotic bound. **Robust**: CodeDiff must handle every file up to the measured maximum, 573,334 AST nodes, with headroom above that number. **Fast**: CodeDiff must compute a diff in under 400 ms for 99.99% of real-world commits,

the range that covers the overwhelming majority of commits per Arafat and Riehle's measurement. The rest of this section reports how well CodeDiff meets both targets in practice, and how accurately it maps the corpus of Section 3.

CodeDiff matches two ASTs in five phases. Each phase either resolves part of the mapping directly or narrows the residual left for the next phase. Later phases only ever see nodes earlier phases left unmatched. A sixth step closes the mapping, recording the deletion or insertion implied by whatever every phase above left undecided.

Phase 1, hash-based descent. CodeDiff matches subtrees by two hash variants, in order: byte-identical subtrees, then same-shape subtrees with any leaf value. Each variant claims what it can before the next runs. This phase is CodeDiff's own design, motivated directly by the commit-size data of Section 3: since most commits touch a small fraction of a file's lines, most of a typical file's AST is byte-identical between before and after. Resolving that identical majority with cheap hashing, before any tree-edit-distance computation runs at all, avoids paying that computation's cost on content that never changed.

Phase 2, contextual exact matching. Comments and modifiers (attributes, decorators) that immediately precede an already-matched node, and diagnostic statements (logging calls, assertions, panics) that are byte-identical on both sides, get matched directly. Both are cheap, high-confidence matches that reduce the work left for the tree-edit-distance phases below. Neither is reachable by phase 1's structural hashing: comments and modifiers are not part of the hashed AST shape, and diagnostic statements are held back deliberately, so that matching one does not fragment a larger match around it.

Phase 3, syntax-aware subtree matching. This phase generalizes three matching strategies into one engine: matching by fully-resolved name (handling overloads and duplicate names across scopes), a greedy cost-estimate matcher for containers with no name structure, such as an `if` block or a loop body, and an overlap matcher pairing import statements that share a base path. Before any of these run, large flat subtrees, such as a macro's token list or a big flat argument list, are pre-matched directly with Myers' algorithm [8], since a flat sequence has no tree structure worth exploiting—exactly the structures that drive the measured AST-size tail of Section 3. Each accepted pair is then handed to APTED [9], scoped to that pair alone, to compute its exact internal edit script. This is the only place a tree-edit-distance computation runs at all, and the scoping is what keeps it affordable: APTED sees one accepted container pair at a time, never the whole file.

Phase 4, residual alignment. Whatever remains unmatched after phases 1 through 3 is resolved by a Myers-LCS alignment [8] over the residual forest, bracketed by two passes of strict bottom-up propagation: a node whose children all match the children of exactly one node on the other side is matched to it, which shrinks the residual the alignment must guess at, and is run again afterward so that pockets the alignment just resolved can still bubble up to their now-complete ancestors.

Earlier versions of CodeDiff instead ran one whole-residual APTED computation here, with the Myers-LCS alignment substituting only when that computation looked too expensive. That design was abandoned: its cost is driven by residual *shape* rather than size, which makes it impossible to gate reliably. Two measured cases make the point—an 11,647-node Vimscript residual took 87 seconds, while a 258,504-node JSON residual took 9.4. No size threshold separates those two, so the exact computation was removed from the whole-file path entirely and survives only in phase 3's scoped form. This is a direct consequence of RA2: an unbounded whole-residual computation cannot meet the 400 ms target at any threshold.

Phase 5, move-detection recovery. The final matching pass. Ordered tree edit distance cannot express a subtree that relocated across a matched boundary as anything but a delete plus an insert. This phase, adapted directly from GumTree's recovery-mappings phase [4], pairs up whole subtrees left fully deleted on one side and fully inserted on the other, when they are byte-identical and large enough to rule out coincidence. It runs last by necessity rather than by preference: every phase above requires some anchor—a hash, a name, a shared matched ancestor—before it will consider a pair at all, and content that moved between two containers that both changed identity has none. It is also, of the pipeline's optional passes, the one whose absence costs the most accuracy: a relocated subtree is the single shape ordered tree edit distance cannot express except as a delete plus an insert, so nothing downstream recovers it.

Zhang-Shasha [10] plays no role in this pipeline as a shipped algorithm. CodeDiff's own test suite instead implements it from scratch as an independent oracle, and fuzz-tests thousands of random trees to confirm that APTED's result always matches Zhang-Shasha's, bit for bit. This checks the pipeline's most correctness-critical component against a second, independently-implemented ground truth, not only against itself.

Accuracy. Measured directly against the human mapping, CodeDiff maps 5,713,065 of 5,719,183 AST nodes correctly across the 512 fixtures, 6,118 mismatches, or 99.89% node-level accuracy. That denominator counts every AST node, including every ancestor of every change up to the root, so it partly reflects how deeply a given grammar nests. Restricted to the 68.3% of nodes RA3.3 finds visible—those carrying text of their own, and therefore able to reach the screen when the diff is rendered—the figure is 99.89%, 4,166 mismatches out of 3,908,441. The two rates agreeing to two decimal places is itself the useful result, and it is exactly the check RA3.3 argues every node-level accuracy claim should carry: CodeDiff's residual errors are not concentrated in the invisible third of the tree, so the headline figure is not being flattered by nodes no reviewer ever sees.

Scored on the same line-level basis as the nine tools of Table 4, CodeDiff marks 4,872 lines differently from the

human mapping over the same 493 fixtures, a rate of 0.795%, below every tool in that table on its own applicable subset. On the 262-fixture subset every tool scored, its rate is 1.265%, which places it above the line-based tools (0.781% for `git`) and below difftastic and GumTree. We report both rather than choose, and the reason they disagree is worth stating, because it is a property of the metric rather than of the subset's language mix. A pooled line rate weights a fixture by its length, and on this subset CodeDiff's errors are extremely concentrated: 5 fixtures hold 3,452 of its 3,866 mismatched lines, 89% of them on 16% of the lines. Set those aside and the remaining 257 fixtures give 0.161% against `git`'s 0.541%. The per-fixture reading does not reorder at all: on the same subset CodeDiff matches the human mapping on every line in 221 fixtures against `git`'s 123, and is closer than `git` on 114 fixtures against 25 the other way.

Those 5 fixtures are not a random tail. Three are long files whose annotations record exactly the phenomena Section 4 measures—one of them the merged call site of Table 3, and one the auto-generated file where the human deletes whole function declarations while a cost-minimizing matcher salvages their braces and keywords into surviving functions, the RA1.2 pattern in its purest form. The other two are open, un-root-caused gaps in CodeDiff recorded as such in the corpus. RA4.1's qualifications apply to every figure in this paragraph exactly as to the others: it is a line-level projection of a node mapping, scored against one of possibly several correct answers.

Speed. Against the tools of Table 6, CodeDiff sits between the two other tree-sitter-based tools at the median, 10.1 ms against diffsitter's 8.6 ms and difftastic's 53.3 ms. Its figure includes parsing, so that it measures the same span as the external tools, each of which is timed as a whole process—temporary file, process start, its own parse and diff, and reading its output back. One asymmetry remains in CodeDiff's favour and we state it rather than carry it silently: measured in-process, it pays no process-startup cost, which every external tool here pays in full. That ordering does not hold across the distribution either. diffsitter, whose narrower hand-picked grammar set does markedly less work, overtakes it at the 90th percentile (63.3 ms against 168.7 ms) and is an order of magnitude ahead at the 99th (239.1 ms against 2259.8 ms); difftastic's tail is heavier still in aggregate cost and essentially ties CodeDiff at the 99th percentile. The line-based tools remain fastest at every percentile, since they never parse or compare tree structure. CodeDiff's tail is heavy, and the design above explains why: the phases that resolve the common case cheaply do nothing for a large, genuinely dissimilar residual, which is exactly what the slowest fixtures in this corpus are.

Read against the Fast target, the percentiles need the population qualification RA4.2 already attaches to the same measurements. The target—400 ms for 99.99% of real-world commits—is a claim about the real distribution of commits, in which 47.5% touch 10 lines or fewer [1], and the fixture corpus is deliberately not that distribution. CodeDiff's 99th percentile on this corpus (2259.8 ms) is therefore a stress figure on an adversarially weighted sample, not the target metric; its median (10.1 ms) and 90th percentile (168.7 ms) on the same hard sample sit well inside the budget. Validating the 99.99% claim on a uniformly drawn commit sample remains future work, and we state it here as a design target rather than a measured result.

Robustness. We sampled 925 real (repository, commit, file) pairs from real Rust repositories and ran every pair through CodeDiff, up to a 16,000 combined AST-node cap and a 120-second timeout per pair. Of the 925 sampled pairs, 377 were unavailable locally because the repository's history had since been rewritten, not a CodeDiff failure, and 142 exceeded the node cap and were skipped before CodeDiff ever ran. CodeDiff completed all 406 remaining pairs with zero panics and zero timeouts. This is a sampled, bounded-scale result, not a claim over the full 100-repository corpus, and the 142 pairs above the node cap are precisely the inputs the Robust target is about, so they are excluded from the evidence rather than counted in its favor. Within those limits the result is consistent with CodeDiff's stated Robust target in this section.

Summary. Combining established techniques rather than inventing new ones, CodeDiff reaches 99.89% AST-node accuracy against human-verified ground truth—99.89% on the nodes a reader can see—the lowest line-level mismatch rate of any tool measured on the full corpus, a median well inside its interactive budget, and zero panics or timeouts across the sampled robustness evaluation. Building a production-grade syntax-aware diffing tool did not require a new matching algorithm, only careful combination and measurement of existing ones, against a problem measured first.

9 RELATED WORK

Tree-diffing tools. GumTree [4] remains the reference point for source-code tree diffing, and CodeDiff's move-recovery phase adapts its recovery-mappings heuristic directly (Section 8). The difference is one of goal: GumTree targets research use, with broad language and backend flexibility, while CodeDiff narrows to a tree-sitter-only frontend and spends the saved generality on production targets. difftastic [5] and diffsitter [3] share CodeDiff's parsing layer but optimize for a diff a human reads comfortably rather than for fidelity of the underlying node mapping; RA4.1 reflects that difference in design goal, not a defect in either tool. GumTree's language and backend flexibility stays valuable well beyond what RA4.1's single corpus measures. Unix `diff` and `git diff`, built on Myers' algorithm [8], remain the baseline every one of these tools must justify itself against—on our corpus they are the fastest tools measured and, at the line level, more accurate than every pre-existing AST-aware tool compared, and BDiff's [7] block awareness does not move that metric either.

Ground truth and its ambiguity. GumTree's own evaluation, and most that followed it, score a tool against a single reference mapping per change. RA1.1, RA3.1 and RA3.2 are an argument that this default understates what a

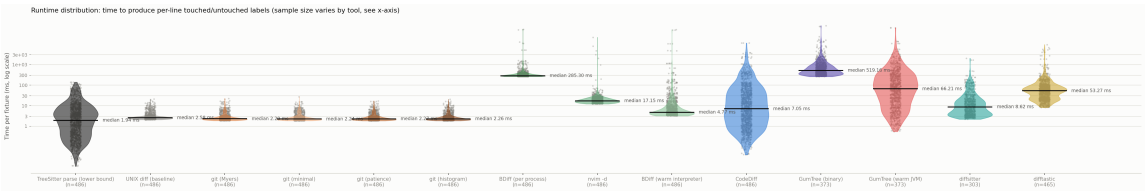

Figure 3: Full per-fixture runtime distribution (log scale), every individual repeated measurement plotted, alongside a tree-sitter-parse-only reference violin (the lower bound every AST-aware tool here must pay before its own work even starts).

corpus can be asked: on the changes where the correspondence is not one-to-one, where several mappings are equally correct, or where several renderings are, a fixed reference records a correct answer as wrong. The corpus here departs from the default in two ways—a multi-map group, which records an N:M site and admits any consistent one-to-one pairing of it, and a second, independently authored painting where the rendering is not unique—and both were added because annotating real changes forced them, not by design. We are not aware of a code-diffing ground truth that records either. RA1.2 adds a third departure from the usual assumptions: the unit cost model that ranks candidate mappings disagrees with the annotators on a measurable minority of changes.

Empirical studies. Dyer et al. [2] and Lämmel et al. [6] mine AST nodes at scale to study API usage; Arafat and Riehle [1] measure the commit-size distribution of open-source software, the distribution CodeDiff's Fast target is stated against. This paper's empirical study differs from these in purpose rather than method: it measures file- and AST-size distributions specifically to set, and then evaluate, the engineering targets of a diffing tool.

10 CONCLUSION AND FUTURE WORK

This paper's most transferable result is not about any tool. It is that the answer a syntax-aware diff is asked to compute is less well defined than the algorithms and metrics assume, in ways measured here without reference to any implementation: the true correspondence is sometimes not one-to-one at all, so no mapping built from insert, delete, update and move expresses it (RA1.1); the cheapest mapping is not always the one humans judge correct (RA1.2); a correct mapping is often not unique (RA3.1); a settled mapping does not settle the rendering (RA3.2); and a third of the nodes a node-level metric scores are invisible to the reader (RA3.3). Underneath all of that, the exact computation every such tool is built on is affordable only for small files (RA2). None of these is a defect in a particular corpus. They are properties of code change, and any evaluation that scores a diff against one fixed answer carries them as an error term whether or not it reports them. Reporting a fixed-answer metric is still defensible; reporting it alone is not.

Against that background, the tool contribution is deliberately modest. None of CodeDiff's matching algorithms are new: it combines APTED's exact tree edit distance, GumTree's move-recovery heuristic, and Myers' sequence diff into a single pipeline, engineered and measured against the real distribution of source code rather than an assumed worst case. On the corpus in this paper that combination reaches an accuracy, speed, and reliability comparable to or exceeding each individual predecessor it draws from, without displacing what any of them are independently good at: GumTree's language and backend flexibility, difftastic's and diffsitter's speed and readability, Unix `diff`'s universality.

The same discipline applies to the exact algorithm at the pipeline's core. CodeDiff began by running one whole-residual APTED computation per file, exactly as its asymptotic analysis invites, and removed it after measurement showed its cost tracks residual shape rather than residual size closely enough that no size threshold can gate it safely. What survives is APTED scoped to individual container pairs, where the same exactness is affordable. An exact algorithm is not automatically the right one to run at whole-file scale, and only measurement distinguishes the two regimes.

Based on the lessons reported in this paper, we recommend the following to builders of syntax-aware developer tooling:

- Report an accuracy rate alongside what its ground truth cannot say: how often that ground truth admits more than one correct answer, and how much of what it scores a reader can see. Both are measurable on any annotated corpus, and both bound the rate being reported.
- Let the annotation format record ambiguity rather than resolve it. Where several answers are equally correct, forcing one makes an arbitrary choice into ground truth, and every tool that chooses differently is then scored wrong for a choice that was never wrong.
- Do not assume the cost-minimal mapping is the one readers want. On a measurable minority of changes the human answer is strictly costlier under unit costs, because it declines to count a coincidental match of identical tokens as a correspondence; an exact optimizer of that cost is exact about the wrong objective there.
- Engineer against the measured distribution of real inputs, not against the worst-case bound alone: the common case and the tail require different machinery, and both occur in practice.
- Scope an exact algorithm to the inputs it can serve within budget rather than abandoning or over-applying

it: the same computation that is unaffordable over a whole file is the right tool for a bounded subtree pair.
- Keep an exact algorithm as an independent test oracle even when the shipped path is heuristic: CodeDiff fuzz-checks its APTED implementation against an independently implemented Zhang-Shasha on thousands of random trees.

Future work includes widening the robustness corpus beyond Rust and extending the accuracy comparison to more languages GumTree also supports, but the clearest directions are the ones RA1.1, RA1.2 and RA3.2 open. The uniqueness rates are floors: the corpus establishes that ambiguity is common, and establishes it only where an annotator noticed and recorded it. Detecting candidate ambiguity automatically, and re-annotating the 213 fixtures that predate the multi-mapping facility, would turn a floor into a census. RA3.2's floor is narrower still and easier to raise: painting is manual, and the 63 changes painted so far are the corpus's small curated ones, so whether 82.5% holds on large, multi-site real-world changes is simply unmeasured. The N:M result needs more than annotation effort. The painted format can express such a correspondence and finds 4.0% of matched regions carrying one; the mapping format records the site but only its one-to-one approximation, and no algorithm in the family measured here can emit anything else, so an output language and a ground truth that carry the correspondence itself are both open. RA1.2 opens the last one: the comparison mapping was a heuristic's, so the rate at which humans prefer a costlier mapping is a lower bound, and a cost model that agrees with readers on those changes is unwritten.

REFERENCES

[1] Osama Arafat and Dirk Riehle. 2009. The Commit Size Distribution of Open Source Software. In *2009 42nd Hawaii International Conference on System Sciences*. 1–8. https://doi.org/10.1109/HICSS.2009.421

[2] Robert Dyer, Hridesh Rajan, Hoan Anh Nguyen, and Tien N. Nguyen. 2014. Mining billions of AST nodes to study actual and potential usage of Java language features. In *Proceedings of the 36th International Conference on Software Engineering (ICSE 2014)*. Association for Computing Machinery, New York, NY, USA, 779–790. https://doi.org/10.1145/2568225.2568295

[3] Afnan Enayet. 2020. diffsitter: A Tree-Sitter-Based AST Difftool for Meaningful Diffs. https://github.com/afnanenayet/diffsitter. Accessed 2026-07-31.

[4] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. 2014. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE '14)*. Västerås, Sweden, 313–324. https://doi.org/10.1145/2642937.2642982

[5] Wilfred Hughes. 2018. Difftastic: A Structural Diff That Understands Syntax. https://github.com/Wilfred/difftastic. Accessed 2026-07-31.

[6] Ralf Lämmel, Ekaterina Pek, and Jürgen Starek. 2011. Large-scale, AST-based API-usage analysis of open-source Java projects. In *Proceedings of the 2011 ACM Symposium on Applied Computing*. 1317–1324.

[7] Yao Lu, Wanwei Liu, Tanghaoran Zhang, Kang Yang, Yang Zhang, Wenyu Xu, Longfei Sun, Xinjun Mao, Shuzheng Gao, and Michael R. Lyu. 2025. BDiff: Block-aware and Accurate Text-based Code Differencing. arXiv preprint arXiv:2510.21094, https://github.com/BDiff/BDiff. Accessed 2026-08-23.

[8] Eugene W. Myers. 1986. An O(ND) Difference Algorithm and Its Variations. *Algorithmica* 1, 2 (1986), 251–266. https://doi.org/10.1007/BF01840446

[9] Mateusz Pawlik and Nikolaus Augsten. 2015. Efficient Computation of the Tree Edit Distance. *ACM Transactions on Database Systems* 40, 1, Article 3 (2015), 3:1–3:40 pages. https://doi.org/10.1145/2699485

[10] Kaizhong Zhang and Dennis Shasha. 1989. Simple Fast Algorithms for the Editing Distance between Trees and Related Problems. *SIAM J. Comput.* 18, 6 (1989), 1245–1262. https://doi.org/10.1137/0218082